

ANALISIS *DOWN SAMPLING* (*MAX, AVERAGE, MEDIUM/MEDIAN*) DAN *UP SAMPLING* (*NN, BILINEAR, BICUBIC*) PADA CITRA DIGITAL

Pengolahan Citra Digital - 1st Assignment

Nama: Andi Nabil Safaraz

NIM: 25/560270/PA/23603

Kelas: KOM A

1. KONFIGURASI DATA

Dalam percobaan ini, untuk mengetahui pengaruh proses *down sampling* dan *up sampling* pada citra digital, Tiga citra yang memiliki karakteristik berbeda digunakan sebagai input yaitu citra laptop dengan latar belakang gelap, citra gradasi warna, dan citra yang memiliki berbagai macam huruf. Ketiga citra memiliki ukuran sama yaitu 256 x 256 piksel. Ketiga citra tersebut kemudian dikonversi menjadi mode RGB agar setiap piksel dapat diproses berdasarkan *channel Red, Green, dan Blue*.



Gambar 1. Tiga citra original yang digunakan dalam percobaan.

Pada Gambar 1, terlihat masing-masing citra memiliki karakteristik yang berbeda. Citra pertama memiliki kontras yang tinggi antara objek terang dan latar belakang gelap. Citra kedua memiliki warna halus/gradasi. Terakhir, citra ketiga mempunyai banyak bentuk kecil, warna, dan *edge* huruf. Tujuannya untuk melihat bagaimana setiap metode baik *down sampling* maupun *up sampling* menangani karakteristik citra yang berbeda.

2. *DOWN SAMPLING*

Implementasi *down sampling* mengurangi jumlah piksel pada citra berukuran 256 x 26 piksel menjadi 128 x 128 piksel pada semua metode. Program awalnya mengambil setiap kelompok 2 x 2 piksel dari citra original kemudian mengubahnya menjadi satu piksel baru.

```
# mengambil 4 pixel dari gambar asli
p1 = gambar.getpixel((x * 2, y * 2))
p2 = gambar.getpixel((x * 2 + 1, y * 2))
p3 = gambar.getpixel((x * 2, y * 2 + 1))
p4 = gambar.getpixel((x * 2 + 1, y * 2 + 1))
```

Gambar 2. Kode mengambil 4 piksel dari gambar asli.

2.1. Metode MAX

```
def downsampling_max(gambar):

    lebar, tinggi = gambar.size

    lebar_baru = lebar // 2
    tinggi_baru = tinggi // 2

    hasil = Image.new("RGB", (lebar_baru, tinggi_baru))

    for y in range(tinggi_baru):
        for x in range(lebar_baru):

            # mengambil 4 pixel dari gambar asli
            p1 = gambar.getpixel((x * 2, y * 2))
            p2 = gambar.getpixel((x * 2 + 1, y * 2))
            p3 = gambar.getpixel((x * 2, y * 2 + 1))
            p4 = gambar.getpixel((x * 2 + 1, y * 2 + 1))

            # mencari nilai terbesar
            r = max(p1[0], p2[0], p3[0], p4[0])
            g = max(p1[1], p2[1], p3[1], p4[1])
            b = max(p1[2], p2[2], p3[2], p4[2])

            hasil.putpixel((x, y), (r, g, b))

    return hasil
```

Gambar 3. Kode metode MAX.

Berdasarkan Gambar 3, metode ini membuat empat piksel berdekatan menjadi satu piksel dengan fungsi `getpixel()`. Kemudian metode ini mencari nilai terbesar masing-masing *channel* RGB menggunakan fungsi `max()` lalu menaruhnya pada hasil citra yang telah direduksi dengan fungsi `put(pixel)`.

2.2. Metode AVERAGE

```
def downsampling_average(gambar):  
  
    lebar, tinggi = gambar.size  
  
    lebar_baru = lebar // 2  
    tinggi_baru = tinggi // 2  
  
    hasil = Image.new("RGB", (lebar_baru, tinggi_baru))  
  
    for y in range(tinggi_baru):  
        for x in range(lebar_baru):  
  
            # mengambil 4 pixel dari gambar asli  
            p1 = gambar.getpixel((x * 2, y * 2))  
            p2 = gambar.getpixel((x * 2 + 1, y * 2))  
            p3 = gambar.getpixel((x * 2, y * 2 + 1))  
            p4 = gambar.getpixel((x * 2 + 1, y * 2 + 1))  
  
            # menghitung rata-rata  
            r = (p1[0] + p2[0] + p3[0] + p4[0]) // 4  
            g = (p1[1] + p2[1] + p3[1] + p4[1]) // 4  
            b = (p1[2] + p2[2] + p3[2] + p4[2]) // 4  
  
            hasil.putpixel((x, y), (r, g, b))  
  
    return hasil
```

Gambar 4. Kode metode AVERAGE.

Berdasarkan Gambar 4, metode ini awalnya sama seperti metode MAX yaitu membuat empat piksel berdekatan menjadi satu piksel dengan fungsi `getpixel()`. Tetapi metode ini mengambil nilai RGB dan dijumlahkan kemudian dibagi empat. Dengan demikian, piksel baru menjadi nilai rata-rata dari area 2×2 tersebut lalu menaruhnya pada hasil citra yang telah direduksi dengan fungsi `put(pixel)`.

2.3. Metode MEDIUM/MEDIAN

```
def cari_median(data): # untuk mencari median

    data = sorted(data)

    nilai_tengah1 = data[1]
    nilai_tengah2 = data[2]

    median = (nilai_tengah1 + nilai_tengah2) // 2

    return median
```

```
def downsampling_median(gambar):

    lebar, tinggi = gambar.size

    lebar_baru = lebar // 2
    tinggi_baru = tinggi // 2

    hasil = Image.new("RGB", (lebar_baru, tinggi_baru))

    for y in range(tinggi_baru):
        for x in range(lebar_baru):

            p1 = gambar.getpixel((x * 2, y * 2))
            p2 = gambar.getpixel((x * 2 + 1, y * 2))
            p3 = gambar.getpixel((x * 2, y * 2 + 1))
            p4 = gambar.getpixel((x * 2 + 1, y * 2 + 1))

            # mengambil nilai R
            data_r = [
                p1[0],
                p2[0],
                p3[0],
                p4[0]
            ]

            # mengambil nilai G
            data_g = [
                p1[1],
                p2[1],
                p3[1],
                p4[1]
            ]

            # mengambil nilai B
            data_b = [
                p1[2],
                p2[2],
                p3[2],
                p4[2]
            ]

            r = cari_median(data_r)
            g = cari_median(data_g)
            b = cari_median(data_b)

            hasil.putpixel((x, y), (r, g, b))

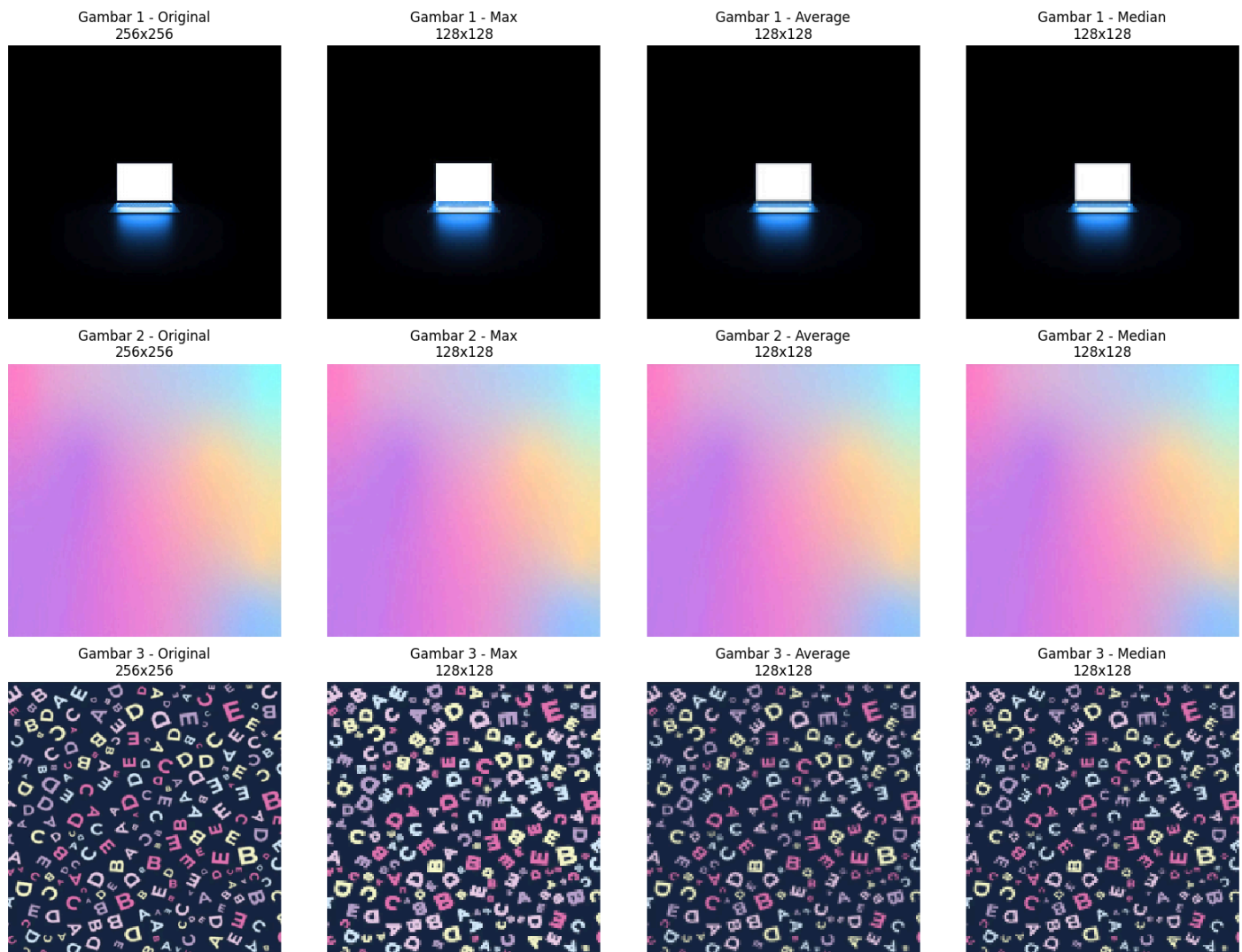
    return hasil
```

Gambar 5. Kode metode MEDIUM/MEDIAN.

Berdasarkan Gambar 5, metode MEDIUM/MEDIAN terlebih dahulu mengurutkan empat nilai pada setiap *channel*. Karena terdapat empat nilai, median diperoleh dari rata-rata dua nilai yang berada di tengah. Metode ini membuat nilai yang terlalu tinggi atau terlalu rendah tidak langsung mendominasi hasil akhir.

2.4. Hasil *Down Sampling*

Setelah semuanya sudah diterapkan, diperoleh hasil seperti pada **Gambar 6**.



Gambar 6. Perbandingan citra original dan hasil *down sampling* menggunakan metode MAX, AVERAGE, dan MEDIUM/MEDIAN.

Kita dapat melihat perbandingan hasil dari karakteristik tiga citra original dengan tiga metode berbeda pada Gambar 6. tersebut. Pada citra laptop, metode MAX menghasilkan bagian terang yang sedikit lebih dominan karena prinsip metode MAX yang selalu memilih nilai RGB terbesar. Pada metode AVERAGE. perubahan warna terlihat lembut karena prinsipnya yang mengambil rata-rata nilai seluruh piksel dalam blok. Pada hasil

MEDIUM/MEDIAN secara visual juga tidak terlalu dipengaruhi oleh satu nilai piksel dan stabil di antara kedua metode yang lain.

Pada citra gradasi, perbedaan antara MAX, AVERAGE, dan MEDIUM/MEDIAN relatif sulit terlihat dikarenakan nilai piksel yang berdekatan sehingga perbedaan pada citra gradasi kecil. Apabila empat piksel dalam satu blok memiliki nilai hampir sama, hasil ketiga metode *down sampling* juga akan memiliki nilai yang hampir sama.

Terakhir, perbedaan lebih mudah dilihat pada citra yang memiliki berbagai macam huruf karena banyak detail kecil, warna berbeda, dan *edge* objek yang rapat. Pada metode MAX, beberapa huruf berwarna terang terlihat dominan dan tebal. Metode AVERAGE menghasilkan tampilan lebih lembut. Kemudian metode MEDIUM/MEDIAN cukup baik mempertahankan bentuk umum pola tanpa menunjukkan nilai piksel ekstrem. Hasil ini menandakan bahwa *down sampling* sangat bergantung pada karakteristik citra sehingga semakin kecil detail gambar, maka semakin mudah kehilangan informasi karena pengurangan jumlah piksel.

3. UP SAMPLING

Implementasi *down sampling* meningkatkan jumlah piksel pada citra berukuran 256 x 26 piksel menjadi 512 x 512 piksel pada semua metode. Berbeda dengan *down sampling*, *up sampling* tidak menggabungkan beberapa piksel menjadi satu namun memerlukan pemilihan nilai untuk piksel-piksel baru yang awalnya tidak tersedia.

3.1. Metode Nearest Neighbor

```
def upsampling_nn(gambar):  
  
    lebar, tinggi = gambar.size  
  
    lebar_baru = lebar * 2  
    tinggi_baru = tinggi * 2  
  
    hasil = Image.new("RGB", (lebar_baru, tinggi_baru))  
  
    for y in range(tinggi_baru):  
        for x in range(lebar_baru):  
  
            x_asal = x // 2  
            y_asal = y // 2  
  
            pixel = gambar.getpixel((x_asal, y_asal))  
  
            hasil.putpixel((x, y), pixel)  
  
    return hasil
```

Gambar 7. Kode metode Nearest Neighbor.

Metode Nearest Neighbor dengan kode terlihat pada Gambar 7. menjelaskan posisi piksel pada citra hasil akan dibagi dua untuk mengetahui posisi piksel terdekat pada citra original. Kemudian nilai dari piksel tersebut langsung digunakan pada citra yang diperbesar. Dengan demikian, metode ini tidak menghitung warna baru namun hanya menggunakan nilai piksel yang sudah ada.

3.2. Metode Bilinear

```
def upsampling_bilinear(gambar):  
  
    lebar, tinggi = gambar.size  
  
    lebar_baru = lebar * 2  
    tinggi_baru = tinggi * 2  
  
    hasil = Image.new("RGB", (lebar_baru, tinggi_baru))  
  
    for y in range(tinggi_baru):  
        for x in range(lebar_baru):  
  
            # mencari posisi pada gambar asli  
            x_asal = x / 2  
            y_asal = y / 2  
  
            x1 = int(x_asal)  
            y1 = int(y_asal)  
  
            x2 = x1 + 1  
            y2 = y1 + 1  
  
            # supaya tidak keluar dari batas gambar  
            if x2 >= lebar:  
                x2 = lebar - 1  
  
            if y2 >= tinggi:  
                y2 = tinggi - 1  
  
            dx = x_asal - x1  
            dy = y_asal - y1  
  
            # mengambil 4 pixel  
            p1 = gambar.getpixel((x1, y1))  
            p2 = gambar.getpixel((x2, y1))  
            p3 = gambar.getpixel((x1, y2))  
            p4 = gambar.getpixel((x2, y2))
```

```

# channel merah
r = int(
    p1[0] * (1 - dx) * (1 - dy)
    + p2[0] * dx * (1 - dy)
    + p3[0] * (1 - dx) * dy
    + p4[0] * dx * dy
)

# channel hijau
g = int(
    p1[1] * (1 - dx) * (1 - dy)
    + p2[1] * dx * (1 - dy)
    + p3[1] * (1 - dx) * dy
    + p4[1] * dx * dy
)

# channel biru
b = int(
    p1[2] * (1 - dx) * (1 - dy)
    + p2[2] * dx * (1 - dy)
    + p3[2] * (1 - dx) * dy
    + p4[2] * dx * dy
)

hasil.putpixel((x, y), (r, g, b))

return hasil

```

Gambar 8. Kode metode Bilinear.

Pada Gambar 8. Kode metode Bilinear, metode ini menggunakan empat piksel di sekitar posisi baru. Dimana program awalnya menentukan posisi koordinat pada gambar original melalui x_asal dan y_asal. Selanjutnya empat piksel terdekat diambil kemudian nilai RGB dihitung menggunakan bobot berdasarkan jarak dari posisi baru. Metode ini menghasilkan perpindahan warna yang lebih halus dibandingkan metode Nearest Neighbor.

3.3. Metode Bicubic

```

def cubic(x):
    x = abs(x)
    if x <= 1:
        hasil = (
            1.5 * x * x * x
            - 2.5 * x * x
            + 1
        )
    elif x < 2:
        hasil = (
            -0.5 * x * x * x
            + 2.5 * x * x
            - 4 * x
            + 2
        )
    else:
        hasil = 0
    return hasil

```

```
def upsampling_bicubic(gambar):
    lebar, tinggi = gambar.size

    lebar_baru = lebar * 2
    tinggi_baru = tinggi * 2

    hasil = Image.new("RGB", (lebar_baru, tinggi_baru))

    for y in range(tinggi_baru):
        for x in range(lebar_baru):

            x_asal = x / 2
            y_asal = y / 2

            x_dasar = int(x_asal)
            y_dasar = int(y_asal)

            total_r = 0
            total_g = 0
            total_b = 0

            total_bobot = 0

            # mengambil area 4 x 4 pixel
            for j in range(-1, 3):
                for i in range(-1, 3):

                    px = x_dasar + i
                    py = y_dasar + j

                    # menjaga supaya koordinat
                    # tidak keluar dari gambar

                    if px < 0:
                        px = 0

                    if py < 0:
                        py = 0

                    if px >= lebar:
                        px = lebar - 1

                    if py >= tinggi:
                        py = tinggi - 1

                    pixel = gambar.getpixel((px, py))
```

```
                    bobot_x = cubic(
                        x_asal - (x_dasar + 1)
                    )

                    bobot_y = cubic(
                        y_asal - (y_dasar + 1)
                    )

                    bobot = bobot_x * bobot_y

                    total_r += pixel[0] * bobot
                    total_g += pixel[1] * bobot
                    total_b += pixel[2] * bobot

                    total_bobot += bobot

            if total_bobot != 0:

                r = int(total_r / total_bobot)
                g = int(total_g / total_bobot)
                b = int(total_b / total_bobot)

            else:

                r = 0
                g = 0
                b = 0

            # nilai RGB harus berada antara 0 - 255

            if r < 0:
                r = 0
            elif r > 255:
                r = 255

            if g < 0:
                g = 0
            elif g > 255:
                g = 255

            if b < 0:
                b = 0
            elif b > 255:
                b = 255

            hasil.putpixel((x, y), (r, g, b))

    return hasil
```

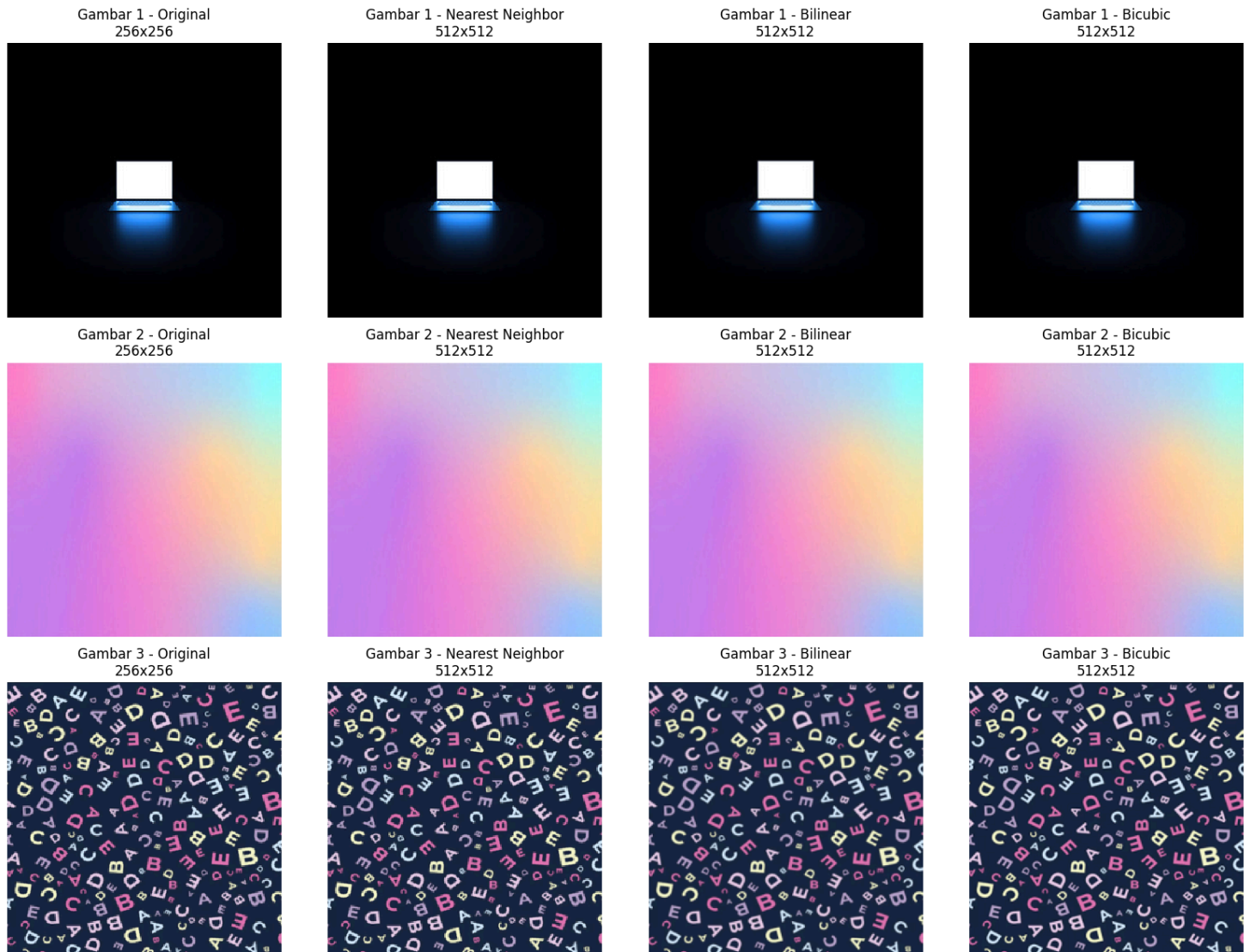
(Baca Dari Kiri ke Kanan)

Gambar 9. Kode metode Bicubic.

Pada Gambar 9. Kode metode Bicubic, prosesnya lebih rumit karena menggunakan area 4 x 4 piksel di sekitar posisi yang dihitung. Program menggunakan fungsi `cubic()` untuk menentukan bobot setiap piksel. Semakin sesuai posisi suatu piksel terhadap titik yang akan dihitung, semakin besar pengaruhnya terhadap nilai akhir. Semua kontribusi R, G, dan B kemudian dijumlahkan dan dibagi berdasarkan total bobot. Nilai RGB hasil juga dibatasi antara 0 sampai 255 agar tetap berada pada rentang nilai warna yang valid.

3.4. Hasil *Up Sampling*

Setelah semuanya sudah diterapkan, diperoleh hasil seperti pada **Gambar 10**.



Gambar 10. Perbandingan citra original dan hasil Up Sampling menggunakan Nearest Neighbor, Bilinear, dan Bicubic.

Kita dapat melihat perbandingan hasil dari karakteristik tiga citra original dengan tiga metode berbeda pada Gambar 10 tersebut. Pada citra laptop, ketiga metode menghasilkan *output* yang mirip jika diamati secara keseluruhan. Namun, perbedaannya dapat dilihat pada bagian tepi layar laptop dan cahaya biru. Nearest Neighbor menggunakan nilai piksel terdekat secara langsung sehingga bagian tepi lebih mempertahankan bentuk piksel original. Metode Bilinear menghasilkan batas lebih halus karena menggunakan empat piksel di sekitarnya, sedangkan metode Bicubic menggunakan lebih banyak piksel sehingga perubahan di sekitar tepi objek dapat terlihat lebih bertahap.

Pada citra gradasi, perbedaan visual ketiga metode juga tidak terlihat besar karena gambar original perubahan warnanya sudah halus. Metode Bilinear dan Bicubic sangat sesuai karena keduanya menghitung nilai baru berdasarkan piksel di sekitar posisi tujuan. Metode Nearest Neighbor tidak melakukan pencampuran nilai warna, tetapi karena perubahan antar piksel pada gambar original sudah kecil, hasil akhirnya tetap terlihat cukup mulus ketika dilihat secara menyeluruh.

Terakhir, perbedaan lebih mudah dilihat pada citra yang memiliki berbagai macam huruf karena banyak detail kecil juga proses interpolasi berpengaruh terhadap tampilan huruf setelah gambar diperbesar. Pada metode Nearest Neighbor, nilai piksel hanya disalin sehingga *edge* objeknya terlihat tajam namun kasar atau *pixelated*. Metode Bilinear menghasilkan tepi lebih halus tetapi beberapa detail masih terlihat sedikit lebih lembut. Pada metode Bicubic terlihat lebih bertahap karena menggunakan 16 piksel. Hasil ini menandakan bahwa walaupun ketiga metode *up sampling* ini berhasil meningkatkan resolusi menjadi 512 x 512 piksel, peningkatan tersebut tidak berarti bahwa informasi baru benar-benar muncul. Piksel tambahan hanya hasil dari perhitungan/penyalinan berdasarkan informasi yang terdapat pada citra original.

4. KESIMPULAN

Berdasarkan analisa dari percobaan yang dilakukan, proses *down sampling* dan *up sampling* memberikan efek yang berbeda terhadap citra digital. *Down sampling* mengurangi resolusi piksel sehingga jumlah informasi yang dapat direpresentasikan menjadi lebih sedikit. MAX mempertahankan nilai intensitas tinggi, AVERAGE menggunakan nilai rata-rata sehingga memberikan hasil yang lebih halus, sedangkan MEDIUM/MEDIAN menggunakan nilai tengah sehingga lebih tahan terhadap nilai yang terlalu ekstrem.

Sebaliknya, *up sampling* meningkatkan resolusi piksel dengan menambahkan piksel baru. Nearest Neighbor hanya menggunakan nilai piksel terdekat. Bilinear menggunakan empat piksel sehingga dapat menghasilkan transisi yang lebih halus, sedangkan Bicubic menggunakan 16 piksel dengan pembobotan yang lebih kompleks. Hasil percobaan juga menunjukkan bahwa perbedaan metode tidak selalu terlihat sama pada setiap citra. Pada citra yang halus seperti gradasi, perbedaannya relatif kecil, sedangkan pada citra dengan banyak batas dan detail seperti pola huruf, pengaruh masing-masing metode lebih mudah diamati.